

ZETROS: A zero-trust IoT network security framework using distributed blacklisting, trust scoring and smart contracts

Cem Ata Baykara ^{a,b,c}, Işıl Şafak ^{d,e}, Kubra Kalkan ^c

^a Medical Data Privacy and Privacy Preserving Machine learning (MDPPML), University of Tübingen, Tübingen, Germany

^b Institute for Bioinformatics and Medical Informatics (IBMI), University of Tübingen, Germany

^c Department of Computer Science, Ozyegin University, Istanbul, Turkey

^d Fibabanka R&D Center, Istanbul, Turkey

^e University of Jyväskylä, Jyväskylä, Finland

ARTICLE INFO

Keywords:

IoT networks
Distributed network security
Machine learning
Trust scoring
IoT security protocols
Smart contracts
Anomaly detection
Zero Trust Model

ABSTRACT

The purpose of Internet of Things (IoT) security is to ensure the availability, confidentiality, and integrity of IoT networks. However, due to the heterogeneity of IoT devices and the possibility of attacks of various kinds from both inside and outside the network, securing an IoT network is a difficult task. Handshake protocols are useful for achieving mutual authentication, which allows secure inclusion of devices into the network. By verifying that the information they receive is accurate and from a trusted source, mutual authentication minimizes the possibility that a malicious actor will compromise their connections. However, handshake protocols do not protect devices from attackers in the network. Use of autonomous anomaly detection and blacklisting prevents nodes with anomalous behavior from joining, re-joining, or remaining in the network. Similarly, trust scoring is another popular method that can be used to increase the resilience of the network against trust based system attacks. In view of the above, the contributions of this paper are three-fold. First, to ensure the security of the IoT network from outsider attacks in a zero-trust environment, we propose a new handshake protocol based on Physical Unclonable Functions that can be used in IoT device discovery and mutual authentication between the IoT device and the server. The proposed protocol is resilient to Man-in-the-Middle, replay and forgery attacks, as proven in our security analysis. Secondly, we propose a real-time intrusion and anomaly detection framework based on machine learning to prevent network-based attacks from insiders. Finally, we propose a trust system which utilizes feedback mechanisms based on smart contracts for managing the trust of a dynamic IoT network to increase resilience against behavioral attacks. Simulation results show that by using blacklisting, our trust management model provides greater resilience against trust-based attacks compared to similar blockchain-based trust models in the literature, and the proposed distributed IoT network security framework can secure an IoT network from both internal and external attacks, even in an environment where half of the devices in the network are compromised.

1. Introduction

Recently, Internet of Things (IoT) has become an integral part of our everyday life since it enables devices to be aware of their surroundings, efficiently communicate, and ultimately create a better networking environment [1]. While these smart devices offer tremendous opportunities, they also cause huge security challenges and risks [2]. These devices have differing computational complexities, and not all of them are equipped with means to mitigate malicious malware. Many IoT devices may even be manufactured with inherent security vulnerabilities [3]. Due to the increasing use of such vulnerable IoT devices

to perform digital transactions, these vulnerabilities pose risks to the security of an entire IoT network [4].

The aim of this paper is to demonstrate that the security of an IoT network can be improved in performing digital transactions against both internal and external attacks in a zero-trust environment. The Zero Trust approach involves eliminating implicit trust and continuously verifying every aspect of digital interactions in order to ensure that an organization is secure [5]. However, ensuring security for all nodes with different computational complexities is a daunting task in a dynamic network where a significant number of nodes are constantly joining and leaving, and the trust of both the clients and servers is

* Correspondence to: Dept. of Computer Science, Sand 14, 72076 Tübingen, Germany.
E-mail address: cem.baykara@uni-tuebingen.de (C.A. Baykara).

<https://doi.org/10.1016/j.comnet.2025.111601>

Received 3 November 2024; Received in revised form 2 July 2025; Accepted 1 August 2025

Available online 19 August 2025

1389-1286/© 2025 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

ambiguous. To protect the security of such a network, the approach needs to be lightweight on the IoT nodes so that even devices with lower computational complexities can be allowed to join the network.

The blockchain-based IoT network security sub-component of the collective intelligence architecture,¹ as described in [6], is validated in this study. Our previous work, presented in [7], proposed a secure architecture for centralized IoT networks that utilized a PUF (Physical Unclonable Function) based handshake protocol and autonomous anomaly detection using machine learning (ML). However, the handshake protocol proposed in [7] has been found to have certain vulnerabilities against replay attacks. While these vulnerabilities do not compromise communication privacy, they do undermine secure authentication. Additionally, the previous system model did not consider the behavioral aspect of malicious devices once they join the network. Taking these factors into account, this paper presents improvements to our previously proposed system model. These enhancements include an improved two-phase PUF based security protocol and a distributed system architecture with a smart contract based feedback mechanism to mitigate behavioral insider attacks.

IoT network simulation results are provided of the proposed security framework. After the devices access the network using the proposed handshake protocol, the CoAP server performs real-time autonomous anomaly detection using pre-trained ML algorithms. IoT nodes that perform transactions through the server are autonomously assigned with trust scores by the smart contract based on their behavior. The proposed security protocol ensures privacy against protocol-based outsider attacks, whereas blacklisting using autonomous anomaly detection and trust management model protects the system from network- and behavior-based insider attacks.

1.1. Our contributions

To the best of our knowledge, the security framework proposed in this paper is the first to address both internal and external security challenges of IoT networks. It provides protection against protocol-, network-, and behavior-based attacks by leveraging physically unclonable functions (PUFs), autonomous and distributed whitelisting/blacklisting, and a smart contract-based trust system. Our key contributions are as follows:

- We propose a security protocol that enables IoT network authorities to establish negotiated communications, provide services, and authenticate devices using PUFs. The protocol also prevents unauthorized devices from joining, rejoining, or remaining in the network, enforcing security in a zero-trust environment.
- We formally analyze the security of the proposed protocol and its authentication mechanism, demonstrating resilience against threats within our model, including Man-in-the-Middle (MITM) attacks, forgery, and replay-based attacks.
- We introduce an ML-based autonomous anomaly detection system that enhances IoT network security against insider network attacks by implementing whitelisting and distributed blacklisting.
- We design a blockchain-based trust management model in which a smart contract continuously updates trust scores for both clients and servers based on their verifiable interactions. This real-time trust evaluation mitigates behavior-based attacks in a zero-trust environment.
- We conduct an IoT network simulation to evaluate the effectiveness of the proposed security protocol, anomaly detection system, and trust management model. The simulation features a CoAP server that implements the proposed protocol and performs distributed anomaly detection using pre-trained ML classifiers. A detailed comparison of different ML classifiers is also provided.

- Our simulation results demonstrate that by integrating blacklisting, the smart contract-based trust management model significantly enhances resilience against trust-based attacks compared to existing blockchain-based trust models in the literature.

1.2. Paper organization

The remainder of this paper is structured as follows. Section 2 background information on key concepts relevant to this study. Section 3 reviews related work and current approaches. Section 4 details the system and adversary models, along with an in-depth description of the proposed framework, including the handshake protocol and its security analysis, the anomaly detection model, and the trust management model. Section 5 presents the performance evaluation of the proposed security framework. Finally, Section 6 concludes the paper.

2. Background

This section briefly introduces the main concepts and technologies used in this paper.

2.1. Constrained Application Protocol (CoAP)

CoAP is a lightweight internet application-layer protocol, suitable for IoT devices, that allows IoT nodes to communicate with the Internet or with each other [8–11]. CoAP provides a request- and response-based interaction model between nodes, while supporting built-in discovery of services and resources. The key features of CoAP, on which the proposed security protocol is implemented, include but are not limited to [11]:

1. Fulfilling machine-to-machine (M2M) requirements in constrained environments.
2. Asynchronous message exchanges.
3. Low header overhead and parsing complexity.

2.2. Physical unclonable functions

PUF is a physical function that, for a given input (challenge), produces an unclonable output (response) that serves as a unique identifier for that specific object [12]. The response generated by the PUF may also be called as the digital fingerprint of that object. This is often achieved by a semiconductor device such as a microprocessor.

The inimitable features of PUFs offer an advantage over other hardware-based security concepts, because, even if the attacker has physical access to the device, it cannot clone its intrinsic properties [12]. Thus, PUFs enable us to perform device identification and authentication in a secure manner. The use of PUFs for cryptographic key generation provides a low-cost alternative when compared with the conventional methods, thus making them suitable for low complexity IoT devices [12]. The security protocol presented in this paper utilizes PUFs to generate the secure session key.

2.3. Trust in computer networks

Due to the ever-increasing size of computer networks and systems, trust is becoming an increasingly important component of network performance [13]. Using the past behavior of an entity in a computer network, the trust score is calculated as a measure of its present behavior [14]. Obtaining reliable and secure evidence and information regarding the interactions and behavior of entities is essential to determine trust scores accurately [15]. Generally, more information and evidence lead to more accurate predictions, also referred to as trust scores.

An entity's trust score can be viewed as providing information about that entity in the IoT network, essentially making the trust system a

¹ EPO patent number EP23155296.9 pending.

valuable tool. Despite the fact that entities within a computer network may calculate their own trust scores, it may be more efficient to rely upon a service provider to do so [15]. The trust management model proposed in this paper relies on smart contracts to transform past behavior and transaction data into trust scores in a Zero Trust network model. In a Zero Trust network model all devices inside the network should not be trusted by default, including central entities.

Trust systems are also subject to trust-based attacks, which can make trust calculations misleading or inconsistent. Therefore, additional measures are needed to increase the robustness of the trust systems against such attacks [16].

2.4. Blockchain

Blockchain is a distributed digital ledger technology that holds information about transactions and other types of data, accessible from the entire network [17,18]. Properties of the blockchain technology such as verifiability (transactions on the blocks can be linked back to the parties involved) and immutability (it is extremely difficult to alter the transaction data once they are recorded in a block) are crucial. Furthermore, once a transaction data is recorded in a block, it immediately becomes transparent and universal. The use of blockchain is an important contribution in this regard, since it will allow distributed trust systems to be more robust and overcome many challenges such as centralization [15].

Ethereum is an implementation of blockchain which connects all participating nodes, providing a peer-to-peer (P2P) network. Transactions on the blockchain are processed by the Ethereum Virtual Machine (EVM), which is also capable of executing smart contracts. Smart contracts are computer programs which run on the Ethereum blockchain [19]. Smart contracts are developed with high-level and Turing-complete languages such as Solidity, which is also the language used in this paper. Once a smart contract is deployed on the blockchain, they can execute on every node of the P2P network and modify their own global state according to their internal logic [20]. Smart contracts operate autonomously based on predefined triggers, which makes them suitable for real-time, transparent, and immutable calculations, such as trust score calculations. Similar to transactions on the blockchain, once a smart contract is deployed it becomes immutable and its internal logic becomes visible to all nodes on the blockchain.

3. Related work

The use of security protocols, blockchain and ML-based security solutions for IoT networks has attracted great interest in recent years. In this section, we summarize some of the work related to our contributions.

3.1. Security protocols

An approach for using PUFs to decrease the computational power in IoT devices is proposed in [21]. This work shows in detail the benefit of using PUFs to establish a secure session with IoT devices by comparing their proposed protocol with other existing solutions such as Datagram Transport Layer Security (DTLS) handshake protocol and User Datagram Protocol (UDP). The results indicate that the authentication process using PUFs achieves a power reduction of up to 45% and requires 12% less memory, compared with the existing solutions listed before. The authors in [21] propose a method of detecting artificially generated challenge-response pairs (CRPs) by using neural networks that their proposed verifier (server) uses to authenticate the nodes. By employing this technique, the server can detect malicious nodes that try to authenticate with the server by using replayed CRPs. However, the protocol can only authenticate existing and trusted nodes in the network; it does not utilize whitelisting or ML techniques for autonomous blacklisting of suspicious devices as proposed in this paper.

In this regard, this protocol is not suitable for general and public use, since it restricts the network's ability to accept new nodes. In contrast, the protocol presented in this paper, while keeping a record of previously explored nodes, allows new nodes to join the network and uses whitelisting and distributed blacklisting techniques in ensuring the security of the IoT network. Lightweight key exchange protocols that use pre-shared secret symmetric keys are proposed in [22,23]. These protocols assume that one or more symmetric keys are readily available to the client and server before the handshake protocol begins. This is evidently not a secure or scalable approach since it requires every new client to obtain the shared keys beforehand and is prone to forgery and brute force attacks. In comparison, the proposed handshake protocol in this paper utilizes challenge response pairs to generate a secure session key and achieve mutual authentication on-the-fly without requiring pre-shared symmetric keys, making it more secure against forgery and brute force attacks, as well as utilizing whitelisting and blacklisting in ensuring network security.

Lightweight key exchange protocols using mutual authentication are proposed in [24,25]. These protocols use PUFs not only for generating session keys, but also for registration and authentication purposes. However, these protocols do not use autonomous, distributed blacklisting techniques in ensuring network security. The protocols store long-term keys to authenticate the server or a device, a process which requires extensive computation. Given the limited computational capability of IoT devices, the proposed protocol in this paper tries to minimize the computation required from the IoT nodes for server authentication with the help of a Certificate Authority (CA). Nevertheless, using the CA has some disadvantages, as it assumes that the CA will always be available, as otherwise the protocol cannot be completed.

Similar handshake protocols are proposed in [26]. These protocols also utilize the low computational complexity of PUFs for IoT devices and thus minimize the requirements for computation, memory, and communication overhead. The results in [26] show that using PUFs for secure session generation is a suitable solution for IoT devices. Another protocol proposed in [27] can generate a secure session between IoT devices. This protocol uses a trusted and pre-authenticated server for secure session generation, similar to the handshake protocol proposed in this paper, which uses a CA for server authentication. However, even though the protocols proposed in [26] can be adapted to real-life use cases, they do not provide a real experimental setup or a real-time simulation to prove this statement. In contrast, the results and details of a real-time network simulation show that the handshake protocol proposed in [27] can easily be adapted to real-life use cases.

Despite its improvements over previous works, the protocol in [27] has certain limitations. It relies on a centralized authentication server that must securely store CRPs, introducing potential scalability issues and a single point of failure. Additionally, while the protocol claims resilience against various attacks, it does not explicitly analyze its resistance to replay attacks or provide a formal security proof under a well-defined adversary model. Moreover, its communication overhead remains relatively high due to multiple exchanges required for key agreement, which may reduce its efficiency in highly constrained IoT environments. Unlike [27], our proposed protocol mitigates these issues by leveraging a distributed approach, implementing a smart contract-based trust system, and incorporating real-time anomaly detection and blacklisting mechanisms to enhance security beyond just authentication.

[28] proposes a secure PUF-based authentication and a certificate free identity-based key exchange protocol suitable for distributed IoT networks. However, whitelisting and blacklisting is not incorporated, and CRPs are not used to establish mutual authentication.

[29] proposes a lightweight mutual authentication protocol using a new public key encryption scheme based on transmitting challenges and checking whether the recipient can respond accordingly. This is similar to the use of CRPs in the protocol proposed in this paper. The protocol is shown to perform significantly better than existing RSA-

and ECC-based protocols. In contrast with the protocol proposed in this paper, this protocol does not require the use of a CA for authentication, does not include whitelisting and blacklisting using ML techniques and real-time trust-scoring based on smart contracts to ensure network security.

[30,31] propose variations of the Datagram Transport Layer Security (DTLS) handshake protocols for IoT networks. [30] proposes a simplified version of the DTLS handshake protocol suitable for IoT devices for a general scenario of end-to-end communications based on software-defined networking (SDN). A controller is utilized in generating a symmetric key dynamically, then encrypting and distributing the key to two communicating IoT devices. Certificate verification is shifted from the IoT device to the more powerful controller, where the controller replaces the DTLS server to make a cookie exchange with the DTLS client. The computational overhead, the energy consumption in the IoT devices and the overall duration of the handshake protocol are shown to reduce. [31] separates the DTLS protocol into the handshake phase and the encryption phase, which is shown to enhance the performance of both the device and the network by using a way to delegate the DTLS handshake phase. The proposed scheme supports secure end-to-end communication despite using delegation. However, neither paper utilizes whitelisting, blacklisting or trust scoring methods in ensuring the network security as proposed in this paper.

A different fingerprinting approach proposed in [32] keeps a log of previously known clients and detect malware and other malicious processes trying to initiate a secure connection using the TLS handshake protocol before the secure session is established. The fingerprinting technique proposed in [32] uses an initial unencrypted hello message sent by the clients after the TCP connection is established. Since this message is unencrypted, TLS fingerprinting extracts metadata presented in the message and generates a fingerprint string using a predefined scheme. After generating a fingerprint, the server maps it to the client by keeping a dictionary of known fingerprints to client mappings. By doing so, the server can detect previously known or blacklisted clients trying to initiate a connection and take actions accordingly. However, it is stated that the TLS fingerprinting technique is not sufficient per se to profile clients effectively. A single fingerprint may map to tens or hundreds of unique clients. Thus, the TLS fingerprinting itself is often a poor indicator and additional information is required to increase its reliability [32]. The handshake protocol presented in this paper uses PUFs and CRPs for both client fingerprinting and session key generation. In order to verify the identity of a client, the server may log the challenges and the session keys used to create sessions and authenticate clients by sending the logged challenges. Since the clients in the proposed handshake protocol use their PUFs to generate the session keys, the client is expected to generate the same session key given the same challenge where only a specific client can create that unique session key due to the uniqueness of the PUF for each client. This approach is both more precise compared to [32], and it does not require additional information about the client for fingerprinting. Hence, the handshake protocol proposed in this paper is more lightweight than TLS fingerprinting technique in [32], which puts additional computational overhead on the server.

3.2. Trust management models

A trust management framework based on verifiable interactions is proposed in [15], which utilizes both blockchain and trust providers, similar to the trust management model proposed in this paper. In their framework, blockchain is used to create the evidence on which trust providers base their trust score assignments, enabling universal, transparent, and verifiable trust scoring. [15] presents an in-depth feedback and evidence-based trust scoring mechanism through the use of evidence mapping and smart contracts. Proof-of-Interaction (PoI) is employed instead of traditional Proof-of-Work (PoW), which is more suitable for low-complexity IoT devices due to the high computational

requirements of PoW. The paper also discusses various trust score calculation methods that can be applied by trust providers within such a system. The proposed system in [15] uses smart contracts in two ways: first, for feedback validation, and second, as an on-chain data storage mechanism. In this model, trust calculations are performed by entities within the network, referred to as trust providers. However, these systems assume that the trust providers are benign and always available to offer trust modeling services. The trust model proposed in this paper eliminates both of these assumptions by delegating the trust computation entirely to smart contracts. Since the internal logic of smart contracts is transparent and immutable, the trust calculations they perform cannot be tampered with and can always be assumed to be legitimate. Moreover, while [15] introduces a robust trust framework, in-depth simulation results involving malicious devices with varying behaviors are not provided, but only briefly discussed.

A framework, called Trust and Reputation Interaction Model (TRIM), is proposed in [33] for modeling trust and reputation systems in both peer-to-peer (P2P) and central server-client (central) networks which include malicious participants. TRIM is proposed as an improved version of traditional Trust and Reputation Systems (TRS). Authors in [33] claim TRS to be vulnerable in a hostile environment and state that trust-based attacks would cause the TRS system to deteriorate. They also underline the lack of formal analysis on the robustness of TRSs against trust attacks due to their generality, expressiveness, or applicability. The proposed TRIM framework mainly involves modeling the interactions between the system's agents. The system uses two different models for interaction (primitive and compact) with various degrees of abstraction. Finally, the model specifies a TRS model and combines it with the interacting agents' model to complete the TRIM structure, which is capable of modeling trust attacks as well as other TRSs. However, secure inclusion of the agents into the environment and protocol- and network-based attack scenarios are not studied. This paper utilizes a security protocol for secure authentication and inclusion of the devices into the network, which is shown to be robust against various protocol-based, and network-based attacks due to use of blacklisting based on ML algorithms.

A trust management model for cloud computing systems is proposed in [34]. The model is capable of dynamically updating the trust scores for each entity in the network and uses the concepts of experience and temporal reputation to calculate the direct trust scores of service providers in the cloud environment. However, the proposed model lacks application and only a theoretical case study is presented to demonstrate its effectiveness. Also, an adversary model and possible attack scenarios are not presented.

A trust model for distributed IoT networks is proposed in [35]. Authors claim that their proposed model, called Context-based Social Trust Model for IoT (CBSTM-IoT), is capable of establishing improved cooperation with trusted nodes and reducing the number of transactions with malicious nodes. The proposed model calculates trust scores based on social relationships between IoT nodes and the context of their interactions. The paper also presents an adversary model to show the resilience of CBSTM-IoT against various trust-based attack scenarios. The limitations of CBSTM-IoT include the computation of trust by each node in the system, which may not be suitable for lower complexity IoT devices. In fact, since the trust of entities inside the system is individually calculated by each node and lower complexity devices require these calculations from other nodes, this introduces further security concerns in an environment with malicious nodes. On the other hand, the trust model proposed in this paper uses smart contracts to autonomously manage and calculate trust within the system, while it also allows individual nodes to calculate the trust of entities within the network themselves, where the information used in trust calculations is publicly accessible via the blockchain.

A blockchain based trust management model for IoT is proposed in [36]. The system architecture proposed in [36] includes multiple

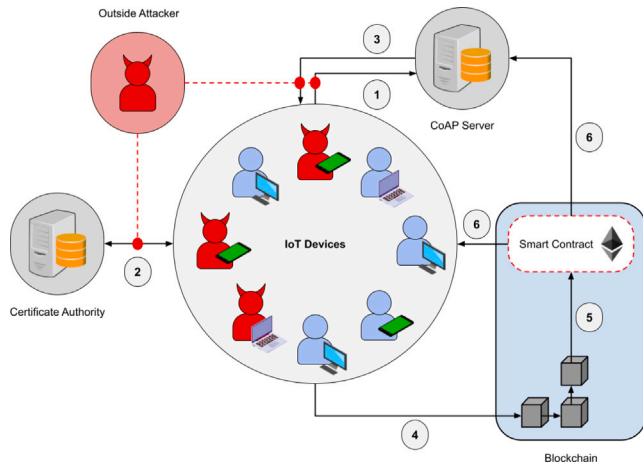


Fig. 1. The proposed network architecture.

manufacturing zones in which devices are managed by an authentication manager. In each zone, IoT devices interact with each other and record their interactions on the blockchain. Recorded interaction data is then used by trust managers to calculate the trust of each device within the network. [36] presents network simulation results which show that by using blockchain technology, trust scores of IoT devices can be efficiently assigned to ensure the security of the network. However, the trust system of [36] does not employ real-time blacklisting which, when coupled with their trust score calculation algorithm, can reduce the security of the network against on off attacks such as opportunistic and collusion based attacks. Through our simulation results, we have shown that by performing real-time blacklisting, our trust model is more resilient against such attacks.

4. Proposed security framework

This section presents detailed descriptions of the proposed framework, the system architecture and the adversary model. Also presented is the proposed security protocol along with a detailed security analysis, the autonomous anomaly detection and blacklisting model based on ML algorithms, and finally the trust management model.

4.1. System model

The proposed network architecture is shown in Fig. 1. The network consists of a CoAP server, an outside attacker, a certificate authority, heterogeneous IoT devices, blockchain, and a smart contract. The proposed system progression is as follows:

1. The CoAP server establishes a secure connection with the IoT devices via the proposed security protocol. The protocol has two phases, namely registration and connection.
2. During both phases of the protocol, the IoT devices authenticate the identity of the CoAP server through a CA.
3. After the devices establish a secure connection with the server using the proposed protocol, they start sending requests to the server. The CoAP server performs blacklisting inside the network using autonomous real-time anomaly detection.
4. After each request, IoT devices score their interaction with the server and record it on the blockchain in the form of feedback. Every feedback is recorded with its relevant evidence. Evidence of an interaction may include any service-related feature provided through an interaction. Since each device inside the network has access to the blockchain, all the feedbacks and their evidence is universal and transparent for all participants inside the network.

5. The smart contract executes when a new interaction is recorded on the blockchain. It checks the feedback and evidence information stored on the blockchain to calculate the trust scores of entities inside the network. The trust scores of both the server and the IoT devices are calculated by and available through the smart contract.
6. The CoAP server and IoT Devices within the network can access the calculated trust scores through the smart contract. Depending on the trust score obtained, devices may decide to stay within the network or leave it, whereas devices with low trust scores may be blacklisted by the server.

4.2. Adversary models

The adversary models considered in this paper include possible attacks from both outside and inside the network. The three main forms of possible attacks considered for the proposed system model are protocol-based attacks, network-based attacks, and trust-based attacks. This section details the default attacker models for each attack type in detail.

4.2.1. Protocol-based attack model

In Fig. 1, the attacker is depicted as an external attacker and embodies the Dolev–Yao attacker model provided in [37]. The attacker can listen, replay, and create messages in the network in order to:

- Generate or obtain the keys used in the session.
- Acquire device information.
- Obtain the confidential information shared during communication.
- Acquire the Challenge Response Pairs generated by the IoT nodes.

Denial of Service (DoS) attacks are not included in the protocol based attack model, but are included in the network-based attack model.

4.2.2. Network-based attack model

After devices establish a secure connection with the server, some devices may generate malicious network traffic within the network. The network-based attacks considered in this paper are as follows:

- Host Brute Force: IoT devices are largely infected by this type of attack using Mirai malware. There are numerous sub-varieties of Mirai malware, but its core concept is to take control of an IoT device by brute force using dictionary-based techniques to guess passwords [38].
- HTTP Flooding: HTTP GET flooding is one of the most effective Distributed Denial of Service (DDoS) attacks. It is performed by flooding the target server with a large number of HTTP requests aimed at slowing down or completely disrupting normal network traffic [39,40].
- UDP Flooding: It is one of the most common forms of DoS and DDoS attacks. In the UDP flood attack, the limited ability of UDP to respond to requests along with the connectionless nature of UDP are exploited. It is performed by submitting a stream of UDP data packets to the target to fill its responding request queue and stop benign users from accessing the target [41].
- Host Port Scan: Port scan attack is performed by sending a TCP or a UDP probe packet to ports of the target host to gain information about which ports are actively listened by the target. Discovered ports may then be used for further attacks [42].
- Host OS Scan: As with a port scan attack, this attack seeks to determine the operating system (OS) of the target machine.

Table 1
Definitions and acronyms.

Acronym	Definition
CA	Certificate Authority
r_C	Random nonce of client
r_S	Random nonce of server
$\oplus$	Exclusive OR operator
ch_S	challenge of server
$+K_{Server}$	Public key of the server
$-K_{Server}$	Private key of the server
$Certificate_{-CK_{CA}(+K_{Server})}$	Certificate of $+K_{Server}$ issued by CA
$CA_{((Server))}$	Short notation of $Certificate_{-CK_{CA}(+K_{Server})}$
$E_K(.)$	Encryption using key K
$D_K(.)$	Decryption using key K
$\theta(.)$	Physical Unclonable Function
$K_{session}^R$	Session Key
$S \xleftarrow{R} S_r$	Random re-ordering of S
$x \in_R S$	Randomly selecting an element from S
C	Challenge set
R	Response set
$\in$	Inclusion
ID_{Client}	Client ID
B	Blacklist
W	Whitelist
$ID_{session}$	session ID
$resp$	Response
I_C	Specifications of the Client
$h(.)$	One-way hash function

4.2.3. Trust-based attack model

Malicious nodes inside the trust system may carry out various attacks to make the trust score calculations inconsistent or misleading. This paper examines attacks on the trust system in the following categories:

- **Explicit Attacks:** These attacks originate from nodes which always give inconsistent feedback in an attempt to bad mouth a legitimate server. As a result, nodes in this group do not actively seek to avoid detection and do not care about their trust in the system.
- **Random Attacks:** These attacks originate from nodes which behave randomly when giving feedback to avoid detection.
- **Opportunistic Attacks:** These attacks originate from nodes that appear benign, providing consistent feedback based on their interactions in an effort to increase their trust ratings. After achieving a certain amount of trust score, or after giving a certain amount of legitimate feedback, they start acting maliciously, such as endorsing a malicious server or bad mouthing a benign server.
- **Collusion Attacks:** These attacks include nodes that act in groups. They pretend to be benign nodes by giving consistent feedback, then suddenly start endorsing a malicious server or bad mouthing a benign server collectively.

4.3. Security protocol

With the proposed protocol, IoT devices can establish a secure connection for communication and mutual authentication using asymmetric encryption, PUFs, and CRPs. Clients and servers that follow the protocol can securely generate a one-time session key to achieve end-to-end encryption for secure communication. Definition of acronyms used within this section are provided in Table 1, and the message sequence charts of each phase are shown in Figs. 2 and 3.

The proposed protocol operates under a Zero Trust model, meaning that no device is inherently trusted at any stage of the process. In this model, implicit trust is eliminated, and all digital interactions are continuously verified to enhance security. As a result, all devices must complete both the registration and connection phases of the protocol to establish a secure session with the server. To authenticate the server, the protocol relies on public key signatures during the registration

phase and on a CA during both phases. Specifically, during the registration phase, the server signs its challenge and nonce values using its private key, enabling the client to verify the authenticity and integrity of the message using the server's public key contained in its certificate. This mechanism mitigates the risk of man-in-the-middle attacks by ensuring that only a server with possession of the legitimate private key can generate a valid signature. The verification process between the device and the CA is assumed to be secure and encrypted, as is standard in cryptographic protocols utilizing Public Key Infrastructure (PKI) and certificate-based authentication [43,44]. Similar assumptions are made in existing secure communication protocols, including TLS-based authentication models, regarding the trustworthiness and security of interactions between devices and CAs [45]. While a detailed analysis of potential vulnerabilities in this verification process falls outside the scope of this study, the proposed model aligns with widely accepted security principles for PKI-based authentication.

The combination of PUF-based key generation and CA-based server authentication ensures both security and efficiency for IoT devices. This is particularly critical for applications such as secure payments, where maintaining communication integrity with minimal computational overhead is essential. Only devices that successfully complete both phases of the protocol are granted access to the IoT network. To further enhance security, the protocol incorporates a blacklisting mechanism to identify and exclude malicious devices. The server employs an anomaly detection model to monitor network traffic in real-time and detect potential threats. Devices exhibiting suspicious behavior are removed from the network and added to the blacklist, preventing them from rejoining or generating further network traffic. By integrating these measures, the proposed protocol provides a lightweight yet robust security solution for IoT communications over CoAP, ensuring that key exchange messages remain within the protocol's efficiency constraints.

4.3.1. Registration

Sequence diagram of the registration phase is shown in Fig. 2. The registration phase is important in two respects. First, it allows the IoT client to obtain a unique ID that can be used by the server for identification. Secondly, it allows the server to log the responses generated by the client's PUF for the challenge set, which can be used for authentication at the connection phase. Detailed explanation for each message is given below.

1. The registration phase is initiated by the client with a hello message which is encrypted with the public key of the server. This first message includes a one-time use nonce called r_C . After receiving the hello message, the server generates its own nonce r_S and calculates a challenge $r_C \oplus r_S = ch_S$.
2. The second message is the server's response to the initial hello message and contains the server's digital signature over certificate issued by the CA, along with r_S , ch_S . Upon receiving this message, the client verifies the signature using the server's public key and then performs a check to ensure that $ch_S = r_C \oplus r_S$, verifying that the server has correctly incorporated the client's nonce. If either check fails, the client terminates the connection immediately.
3. After verifying that the server's response is authentic and correctly formed, the client sends the certificate of the server to the CA for validation. This message is encrypted with the public key of the CA and includes a second nonce r_{C_1} generated by the client to prove liveness of the communication.
4. If the CA verifies the certificate, it sends an *OK* message on message 4. If the certificate cannot be verified, the CA responds with a *NOK* message. In this case, the client terminates the connection as the server cannot be trusted.
5. If the client receives an *OK* message, it then uses its PUF to generate a one-time use session key as $\theta(ch_S) = K_{session}$. Message 5 is encrypted with the $+K_{server}$ and it includes the generated $K_{session}$ alongside the ch_S to prevent replay attacks.

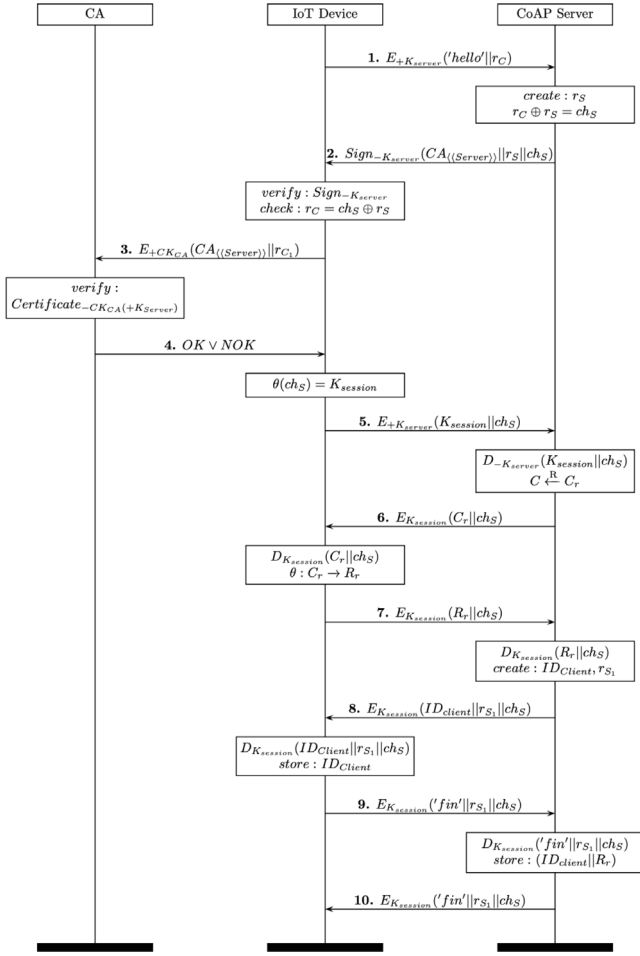


Fig. 2. Message sequence chart for the registration phase.

6. The server decrypts the 5th message using its private key $-K_{server}$ to obtain $K_{session}$. The server uses a challenge set C (which can be updated at pre-defined time intervals) to authenticate the IoT devices. Since any device can acquire the challenges in the C by initiating the registration phase, a way to prevent the attackers from mapping the challenges to the responses generated by the IoT devices is required to prevent replay and forgery attacks. The server achieves this by randomizing the order of the challenges every time an IoT device initiates the registration phase as $C \xleftarrow{R} C_r$. After creating a randomly ordered C_r , the server symmetrically encrypts it with $K_{session}$ and sends it to client on message 6. This message also includes the initially generated ch_s to prevent replay attacks.
7. After decrypting message 6 using $K_{session}$ and obtaining the randomized C_r , the client uses its PUF to generate a response set R_r for each challenge in the order they appear on C_r as $\theta : C_r \rightarrow R_r$. The client then encrypts the R_r using the $K_{session}$ and sends it to the server on message 7.
8. After receiving the response set sent by the client the server creates a unique ID for the client and a second nonce r_{S_1} . It then sends these back to the client on message 8. The server does not store R_r and ID_{client} yet because the registration phase is still not complete and the client may still drop out of the protocol.
9. When the client receives the message 8, it sends message 9 to the server as a finished message. Similar to message 8, message 9 includes r_{S_1} and ch_s .

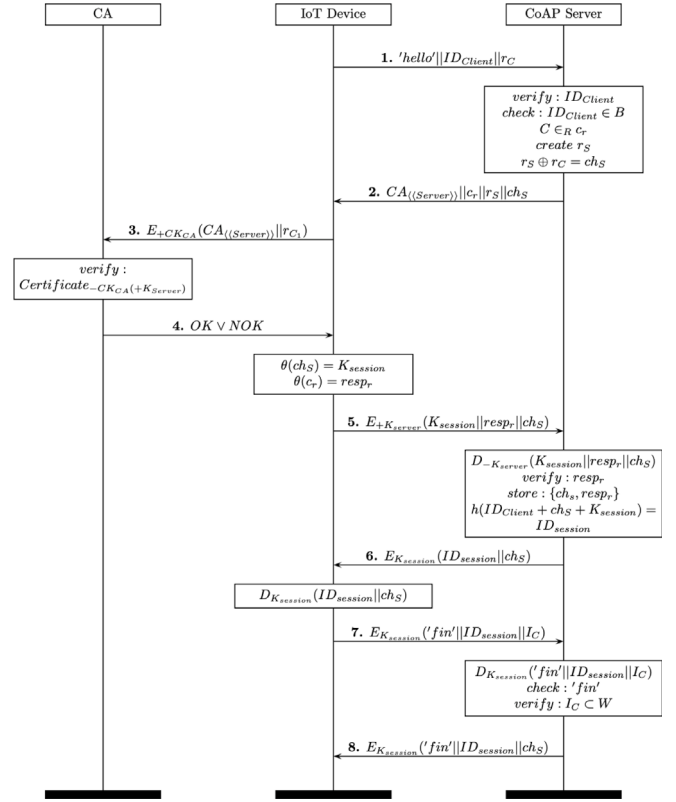


Fig. 3. Message sequence chart for the connection phase.

10. After receiving the client's finished message, the server stores ID_{Client} and R_r . The server completes the registration phase by sending the last finished message in message 10.

4.3.2. Connection

In the connection phase, IoT devices that have previously completed the registration phase and received their client IDs can establish a secure session with the server by authenticating themselves using their PUFs. The connection phase also allows the server to obtain information about the specifications of the IoT devices joining the network for both whitelisting and device discovery purposes.

A detailed explanation for each message is given below (see Fig. 3):

1. Following the completion of the registration phase, a client initiates the connection phase by sending a hello message (message 1) with the unique client identifier ID_{Client} . Alongside the ID_{Client} this message also includes a random nonce r_C .
2. After receiving the hello message, the server checks its validity and checks if it has been blacklisted before. If the ID_{Client} passes both checks, the server then creates a random nonce r_s to calculate a challenge as $r_c \oplus r_s = ch_s$, and also randomly selects a challenge from C as $C \in R C_r$. The server then sends a message back to the client (message 2) which includes its certificate issued by the CA, c_r , r_s , and ch_s . Sending r_s unencrypted poses no security concerns as all the challenges in C can be obtained by any client through the registration phase.
3. Similar to the registration phase, the client authenticates the server through the CA by sending the server's certificate in message 3.
4. After the CA verifies the certificate, it sends an OK message on message 4. If the certificate cannot be verified, the CA responds with a NOK message, in which case the client terminates the connection, since the server cannot be trusted.

5. If the certificate of the server can be verified and the client receives an *OK* message from the CA, then the client uses its PUF to generate both the session key and the response $resp_r$. The response generated will be the same as the response generated during the registration phase due to the properties of PUF, which allows the server to authenticate the client. After generating the response, the client encrypts it with $+K_{server}$ and sends it to the server on message 5 alongside $K_{session}$ and ch_S .
6. After decrypting message 5 using its $-K_{server}$, the server checks the response of the client against the response generated during the registration phase. A same response means that the client can be successfully authenticated while a different response may indicate that the client may be trying to use the ID_{Client} of another client. If the responses do not match, the connection is terminated immediately by the server. Multiple failed connection attempts from the same devices can be subjected to a timeout to prevent server overload. If the responses match, the server stores ch_S and $K_{session}$ together, to be used at a later time to authenticate the same device. It also creates a unique session ID for the secure session that will be started shortly using a one-way hash function $h(ID_{Client} + ch_S + K_{session}) = ID_{session}$, which is sent to the client on message 6 alongside ch_S and encrypted with the $K_{session}$.
7. After receiving and decrypting message 6, the client replies to the server with a finished message, which also includes its specifications.
8. After the server receives and decrypts message 7, it checks the specifications of the device against the whitelist. If the device is eligible to join the network, the server sends a final finished message back to the client on message 8. The secure symmetric encryption has now been achieved by both client and server, and the ensuing session is continued with K_S as the symmetric key.

4.3.3. Security analysis

Based on the model described above, the security analysis of the protocol is provided in this section. The likelihood of an attacker breaking the security guarantees of the protocol to achieve various goals is examined. The analysis is based on the following assumptions:

1. The signature scheme used by the protocol participants (CoAP server, IoT client, and CA) is secure. This implies that it is impossible for an attacker to forge the signature without the private key.
2. PUFs used by the IoT devices to generate responses are based on intrinsic qualities of their hardware and therefore cannot be manipulated and/or stolen.
3. The communication between the IoT devices and the CA is secure.
4. The challenge set C contains a sufficient number of challenges so that it is impossible for an attacker to predict different possible sequence combinations.
5. Random nonces generated by the client and the CoAP server are never picked twice.

Based on these assumptions, the proposed security protocol provides four guarantees.

Guarantee 1 For the server, the protocol guarantees authentication with the legitimate owner of any ID_{Client} in the connection phase.

Proof. An attacker may try to authenticate using the ID_{Client} of another client since it is shared unencrypted on the first message of the connection phase. An attacker has two options to authenticate using the ID_{Client} of another client. The attacker should either be able to provide the same response $resp_r$ generated by the client to the randomly selected c_r or replay a message which includes the response of the client generated from the same c_r . Since the client is using a PUF,

the attacker cannot generate the same client-generated response for r_r . Based on assumption 2, PUF cannot be replicated or manipulated since it is dependent upon the intrinsic qualities of the hardware of the device. Therefore, unless the attacker has physical access to the device itself, it is impossible for him to generate the same response. The other option for the attacker to authenticate as another client is to replay an already generated $resp_r$ for c_r . To achieve this, the attacker is required to capture and replay the message 5 on the connection phase which must include a response generated from the same c_r . Since c_r is sent unencrypted on message 2 of the connection phase, the attacker may wait for it to be selected again and then replay the same message to authenticate. This is not possible due to both assumption 2, assumption 5. Since message 5 includes the $K_{session}$, which is generated by a PUF as $\theta(ch_S) = K_{session}$, and since it is impossible for the attacker to predict the behavior of a PUF without access to the device itself, the attacker can never produce the same $K_{session}$ again, regardless of his knowledge of ch_S . Even if the attacker gained access to the $K_{session}$, a replay attack would still be impossible because message 5 also contains ch_S , and $K_{session}$ is produced using ch_S , and assumption 5 states that nonces cannot be picked twice.

Guarantee 2 For clients, the protocol guarantees authentication with the original server.

Proof. If an attacker creates a fake server in registration and/or connection phase, he must provide clients trying to connect to his server with a signed certificate in message 2 of both phases to proceed with the security protocol. Due to the fact that a client verifies the identity of the server using the certificate authority, an attacker may pose as the real server by creating or replaying an *OK* message on message 4 during registration or connection phases after the client presents the attacker's certificate in message 3. The attacker can never generate message 4 for an illegitimate certificate because of assumption 1, which states that the signature scheme used by the protocol participants is secure and it is impossible to forge a signature without the private key. Thus, the attacker cannot forge an *OK* message unless he has access to the private key of the CA which is never shared. Another option to authenticate as a fake server is to replay an *OK* message, which is also impossible because assumption 3 states that the communication between the CA and the client is secured. The security of the communication between the CA and the client implied in this assumption also means that the connection is live. Therefore, an attacker can never replay an *OK* message as, after encryption, the same *OK* message is never generated again.

Guarantee 3 The protocol guarantees the security and confidentiality of the session generated after the connection phase.

Proof. The attacker needs to either forge or replay message 5 of the connection phase to break the security and confidentiality of the session generated between the IoT client and the CoAP server. To forge the session key sent on message 7, the attacker would need the θ function used by the client. This is not possible without having physical access to the device because of assumption 2. It is also impossible for the attacker to replay the message containing $K_{session}$ because the message also contains $resp_r$ and ch_S . assumption 2, assumption 5 prevent the attacker from generating or replaying $resp_r$, and assumption 5 prevents the attacker from replaying ch_S .

Guarantee 4 The protocol guarantees that the pairing of the response set R_r generated by the client to the challenge set C_r , where $C \xrightarrow{R} C_r$, is known only by the client and the server.

Proof. One way an attacker can obtain clients' challenge-response pairs is to pretend to be a legitimate server and give clients a C_r whose order is known by the attacker. This is not possible because of assumption 3 and the reasons explained in guarantee 2. Another way to obtain

the challenge-response pairs is to produce the response set R_r for a given C_r , which is also not possible without physical access to the device itself because of assumption 2. Since assumption 4 states that C contains a sufficient number of challenges, the number of possible sequence combinations of a challenge set C increases very rapidly with increasing challenges due to combinatorial growth. Lastly, challenge-response pairs of a client can be obtained by breaking the encryption of messages 6 and 7 of the registration phase, which is only possible by knowing the $K_{session}$. $K_{session}$ can only be obtained by either forging it by knowing the ch_S or by decrypting message 5, which is the only message where it is shared. Because of assumption 2, $K_{session}$ cannot be produced by the attacker, even if he knows the ch_S . Therefore, the only other way to obtain the $K_{session}$ is to decrypt message 5, which is impossible because of assumption 1.

4.4. Anomaly detection and blacklisting model

The proposed security protocol ensures mutual authentication and communication privacy but does not prevent non-black-listed devices from entering the network. To mitigate potential attacks within the network, we introduce a blacklisting approach that utilizes autonomous anomaly detection through machine learning. Anomaly detection is performed by the CoAP server on network traffic generated after devices establish a secure session using the proposed protocol. Devices that generate abnormal or malicious network traffic are detected and subsequently blacklisted by the server. Blacklisted devices are then prevented from joining, re-joining, or remaining in the network.

While the anomaly detection model itself is based on established ML techniques, its novelty lies in its integration within a broader security framework. Rather than presenting a novel anomaly detection algorithm, this paper focuses on the application of existing ML classifiers in the context of a comprehensive IoT security model. The integration of anomaly detection within the security framework enhances its overall effectiveness, particularly in enabling real-time blacklisting of malicious nodes.

The ML classifiers considered in this paper have been extensively studied in the literature for intrusion and anomaly detection [46,47]. The classifiers evaluated include:

- Random Forest (RF)
- Decision Tree (DT)
- Stacking
- K-Nearest Neighbor (KNN)
- Gaussian Naive Bayes (GNB)
- XGBoost

DT, KNN, and GNB classifiers are preferred due to their suitability for intrusion and anomaly detection [47,48]. In our evaluation, Random Forest, Stacking, and XGBoost classifiers were found to perform particularly well on the IoTID20 [49] and CIC-IDS2018 [50] datasets used for network simulation. More details about the datasets are provided in Section 4.

4.5. Trust management model

The trust management model implemented in this paper is similar to the framework proposed in [15]. In our proposed network system model, the CoAP server is responsible for securing IoT devices by performing real-time anomaly detection and blacklisting. However, since the proposed network model does not initially assume reliable servers, the CoAP server itself may be malicious and cannot be initially trusted. Therefore, clients collectively rate their interaction with the server based on different parameters of that interaction. IoT devices record their feedback on the blockchain so that trust scores can be calculated by the smart contract. Participating nodes can then request the current trust scores by interacting with the smart contract directly.

The proposed system model is flexible, allowing the devices to calculate the server's trust score themselves if they do not want to use the trust service provided by the smart contract, as all feedback and historical interaction data is universally available through the blockchain. The use of blockchain is also beneficial in a system architecture where multiple trust providers are present as it allows them to base their calculations on the same feedback and evidence set [15]. The aim is to improve blacklisting accuracy where both malicious clients and servers are present.

The proposed system model for the trust management model consists of a CoAP server, the blockchain, a smart contract and multiple IoT devices. Devices connect to the CoAP server by going through the proposed security protocol. Devices that enter the network start sending requests to the server and, based on the features of this interaction, they leave feedback on the blockchain. Each feedback saved on blockchain also includes various features, the so-called evidence, of the interaction between the server and the client. These features as a whole represent the quality of the service provided. This paper assumes that the evidence linked to a feedback score cannot be manipulated. The smart contract uses the evidence and feedback score recorded on the blockchain to calculate the trust scores of both the server and the clients. The trust score of the server is based on the evidence and the feedback recorded on the blockchain, whereas the trust score of the individual clients is calculated based on the consistency of their feedback with their evidence.

To address privacy concerns regarding the potential linkability of feedback to individual devices, we anonymize the identities of IoT clients using Decentralized Identifiers (DIDs) compliant with the Ethereum ERC-1056 standard. Instead of recording real-world identifiers on the blockchain, each client is associated with a DID generated via the ethr-did library [51]. This pseudonymous identity allows clients to interact with the smart contract without revealing their actual identity. Thus, the recorded feedback and evidence are unlinkable to specific real-world devices, ensuring that trust scores cannot be traced back to individual entities.

Trust can be expressed both quantitatively and qualitatively. In this paper, trust scores are calculated based on solid evidence in the form of past behavior and interaction. Therefore, the concept of trust score is limited to measurable trust in this study. An ideal system model would have multiple smart contracts performing trust score calculations using different algorithms, which adds diversity and increases the robustness of trust score calculations. However, in this paper, for ease of simulation, uses only a single smart contract which calculates the trust score using a set algorithm. Since the trust calculation is done by a single entity, the algorithm used to obtain accurate and reliable results becomes important.

To determine the trust score calculation algorithm to be used, we examined implementations in a framework that supports various trust calculations provided by [15]. In [15], averaging is claimed to be an excellent method for calculating the trust score, since it is reliable when there is a sufficient amount of feedback, and more importantly, it requires significant effort to make major modifications. Averaging is used to calculate the trust of the clients based on the consistency of their feedback and the evidence of the interaction. Assume a continuous satisfaction feedback function $f : X \rightarrow [0, 1]$ that links an interaction to its feedback, where 1 is very satisfied and 0 is completely dissatisfied. A continuous consistency function $\phi : F, E \rightarrow [0, 1]$ is used to check the consistency of the feedback f against its evidence e . Let n denote the total number of feedbacks provided by a client and β shows the feedback trace on the blockchain belonging to that client. According to the trust scoring algorithm based on averaging, the trust score η of a client is defined by:

$$\eta(\beta) = \frac{\sum_i^n \phi(f, e)}{n} \quad (1)$$

In this paper, the calculation of the trust score of the server is based on weighted averaging. Here α denotes the feedback data available

on the blockchain provided by all clients. In a setting where the total number of clients in the network is I , each client has a trust of w as calculated by the smart contract. The trust score μ of the server is defined by:

$$\mu(\alpha) = \frac{\sum_i^I (\sum_i^n \frac{\phi(f,e)}{n} \cdot w_i)}{\sum_i^I w_i} \quad (2)$$

A different weighted averaging is presented in [36] to define the trust relationship between two entities, namely the trustor e_i and the trustee e_j , at any given time as $t : trust(e_i \mapsto e_j)$. The trust value, denoted by $\hat{T}_{ij}(t)$, provides a measure of the trust of an entity e_i has towards e_j at time t . In this formulation, $\hat{T}_{ij}(t)$ is calculated by using both the direct trust, the immediate trust between e_i and e_j , and the indirect trust, the trust of other nodes towards e_j . The trust value between two entities is computed as a weighted average of the most recent trust value, and the past trust evaluations during a specific time interval Δt as [36]:

$$\hat{T}_{ij}(t) = w1 * \hat{T}_{ij}(t - 1) + w2 * \hat{T}_{ij}(\Delta t) \quad (3)$$

where the weight factors $w1$ and $w2$ satisfy $w1 + w2 = 1$, and $0 \leq w1, w2 \leq 1$. Previous trust values are also included because behaviors of entities inside the network may vary with time [36]. However, [36] does not discuss how $w1$ and $w2$ should be selected, which represents a significant optimization challenge. A network without blacklisting may allow entities with a track record of malicious behavior in the past to obtain a high trust value at t if $w1$ is chosen to be too high. Furthermore, choosing an optimal Δt is also a challenging task for long-lived networks. For the simulations performed in [36], $\Delta t = 1800s$ was used.

Using the trust score evaluation presented in [36], we demonstrate in Section 5.5 that malicious entities performing collusion attacks can achieve high trust scores without blacklisting.

5. Performance evaluation

This section presents the results of the IoT network simulation performed to measure the effectiveness of the proposed security framework.

Network simulation performed in this paper includes the simulation of the proposed secure protocol, the autonomous real-time anomaly detection and blacklisting, and the behavior of both anomalous and normal IoT nodes and the server in the blockchain network with their trust score assignments performed by the smart contract. The simulation is performed using Python 3.7.

A description of the simulation environment is provided in this section as well as the results of the simulation. Additionally, it discusses the datasets used to train the ML classifiers, the relative performances of various ML classifiers based on their performance and suitability for real-time anomaly detection, and the performance of the proposed trust management model against various trust-based attacks.

5.1. Datasets

The IoTID20 dataset is used to simulate the traffic in an IoT network [49]. The data is generated using a smart home environment which includes both benign and malicious devices. The dataset contains over 625,000 data instances which consist of 80 network features and 3 label features, which are related to the categories shown in Table 2.

The IoTID20 dataset is reported to provide a good representation of modern-day IoT device communications [49]. The dataset is also among the few publicly available IoT intrusion detection datasets. The features inside the data are ranked using the Shapiro-Wilk algorithm to measure the regularity of the instance distribution based on their features. Based on this analysis, it is claimed that more than 70% of the features ranked above 0.5, and these high-ranked features will improve the classification capability of ML classifiers and techniques.

Table 2

Label features of the IoTID20 dataset.

Binary	Category	Sub-Category
Normal	Normal	Normal
Anomaly	DoS (Denial of Service)	Syn Flooding
	Mirai	Brute Force
	MITM (Man in the Middle)	HTTP Flooding
	Scan	UDP Flooding
		ARP Spoofing
		Scan Host Port
		Scan OS

Table 3

Label features of the CICIDS-2018 dataset.

Binary	Category
Normal	Normal
Anomaly	Bots
	Brute Force
	DDoS
	DoS
	Infiltration
	Web Attacks

The subset of data used for the network traffic simulation in this paper is generated by randomly selecting 20,000 instances from each of the five main categories shown in Table 2. The data is revised to be compatible with the CoAP protocol, and additional features are added for ease of simulation. However, these features are not used by the classifiers during the training phase.

To further evaluate the generalization and real-time anomaly detection capabilities of the proposed method, we additionally include the CICIDS-2018² [50] dataset. This dataset is an extended version of the older CICIDS-2017 dataset and contains a comprehensive set of network traffic instances, collected from a simulated enterprise environment, capturing both benign behavior and a variety of contemporary cyber-attacks. The dataset is well-suited for evaluating intrusion detection systems due to its diverse attack landscape and realistic traffic patterns.

We followed similar preprocessing steps as outlined in [52]. In our experiments, we focus on data collected over multiple days, retaining a balanced subset of normal and anomalous instances. The raw data is cleaned by removing constant, sparse and highly correlated features, and only numerical attributes are retained for compatibility with our learning models. All features are standardized using z-score normalization. For label construction, we group the attack types into broader categories used for model training, as summarized in Table 3.

Further pre-processing was performed to address class imbalance and to ensure temporal consistency for real-time evaluation. We sampled an equal number of benign and attack instances per day, preserving the original time ordering. Some categorical attributes such as protocol were one-hot encoded to maintain compatibility with models expecting continuous input.

5.2. Performance evaluation of the classifiers

Tables 4 and 5 show the relative performances of the ML classifiers trained on the IoTID20 data based on their accuracy, precision, recall and F-1 score. The classifiers RF, DT and Stacking perform well on the IoTID20 data set. Although KNN classifiers perform similarly to RF and DT classifiers, [53] argues the algorithm may not be suitable for critical real-time systems with high training instances. Additionally, the KNN algorithm requires a significant amount of computational resources since it needs to calculate the distances between each new sample and the entire training space. Lastly, the GNB classifier performs the worst

² <https://registry.opendata.aws/cse-cic-ids2018/>

Table 4

Macro Avg. Performances of the classifiers trained on category as target label on the IoTID20 dataset.

Classifier	Accuracy	Precision	Recall	F-1 Score
RF	0.90	0.93	0.90	0.90
DT	0.90	0.93	0.91	0.91
Stacking	0.91	0.93	0.92	0.92
KNN	0.87	0.91	0.87	0.86
GNB	0.56	0.66	0.56	0.56

Table 5

Macro Avg. Performances of the classifiers trained on sub-category as target label on the IoTID20 dataset.

Classifier	Accuracy	Precision	Recall	F-1 Score
RF	0.82	0.73	0.74	0.71
DT	0.85	0.78	0.77	0.74
Stacking	0.89	0.86	0.85	0.84
KNN	0.78	0.71	0.72	0.69
GNB	0.48	0.46	0.43	0.40

Table 6

Macro Avg. Performances of the classifiers trained on CICIDS-2018 dataset.

Classifier	Accuracy	Precision	Recall	F-1 Score
RF	0.97	0.87	0.87	0.87
DT	0.93	0.80	0.89	0.83
KNN	0.92	0.78	0.87	0.81
XGBoost	0.97	0.88	0.86	0.87

among the considered algorithms, since it achieves an accuracy score of 56% in category and 48% in sub-category labels, respectively. Its performance is 35% and 41% lower than the best performing Stacking classifier in the category and sub-category labels, respectively.

In this paper, the Stacking classifier is preferred for the network simulation since it has a higher performance compared to other classifiers. However, as an ensemble classifier, the Stacking classifier requires significantly more computational power and resources than the other classifiers.

The classification time of the algorithms are considered for real-time use. The Stacking classifier takes 0.06 s to classify a single instance, whereas the second best performing DT classifier takes 0.01 s. The stacking algorithm is still preferred since a difference of 0.05 s is negligible for the network simulation performed in this paper. However, CoAP servers using the Stacking classifier may experience problems detecting anomalous network traffic in real-time if the network traffic increases significantly. In such cases, the DT algorithm should be preferred.

To further evaluate the generalizability and robustness of the proposed anomaly detection framework, we extended our experiments to include the CICIDS-2018 dataset [50], a more comprehensive and up-to-date benchmark for intrusion detection. Following similar pre-processing steps outlined in [52], we trained and evaluated multiple classifiers, RF, DT, KNN, and XGBoost. As summarized in Table 6, all classifiers achieved strong performance on the dataset, with RF and XGBoost obtaining the highest accuracy (0.97) and F-1 scores (0.87). DT and KNN also performed well, though with slightly lower precision and overall F-1 scores.

In the baseline analysis, XGBoost was recommended for real-time anomaly detection due to its balance of accuracy and computational efficiency [52]. In line with this recommendation and supported by our own performance results on the CICIDS-2018 dataset, we also adopt XGBoost as the preferred classifier for real-time detection in our simulation with the CICIDS-2018 data. Compared to the other models evaluated, XGBoost provides strong predictive performance while maintaining scalability under increased network traffic, making it well-suited for deployment in real-world IoT scenarios.

5.3. Simulation of the security protocol

The CoAPthon v4.0 [54] library is used as a basis for simulating the CoAP protocol, which we have modified to both better fit other libraries used in the simulation environment and to address some implementation issues. The Average latency of the registration phase, the connection phase, and the TLS protocol are compared in Table 8. It is important to note that the latency of the TLS protocol is measured in a simulation environment but not in a real-life scenario. Therefore, the latency of the TLS protocol may be expected to increase due to external factors such as distance between the participants.

One may thus conclude that, with an additional 280 ms latency, introduced by the registration phase on the first connection only, on top of the latency required by the TLS protocol, our security protocol can provide both mutual authentication by the participants, and whitelisting and blacklisting.

5.4. Simulation of the anomaly detection model

An IoT network is simulated using a central CoAP server, another server acting as a CA, and 30 IoT devices acting as clients. The IoT devices range from simple and lower complexity devices such as smart cameras, controllers and hubs, smoke alarms and printers, to more complex devices such as laptops, PCs, phones, and tablets. It is assumed that none of these devices are previously known by the CoAP server. Therefore, each of these devices perform the registration phase to obtain a unique client ID and then use the connection phase to establish a secure connection with the CoAP server.

A total of 30 IoT devices, which are used to test the effectiveness of both the security protocol as well as the anomaly detection model, consists of 25 victim and 5 malicious devices. All malicious devices are assumed to be high complexity devices. The numbers, roles and computational complexities of the devices used in the network simulation are shown in Table 9.

After going through both phases of the proposed security protocol and establishing a secure session with the CoAP server, each device starts generating a network traffic. Each request sent by the IoT devices was chosen randomly from within the test data as discussed in the dataset section. For ease of implementation and simulation, the information within these packets is known by the CoAP server. This information is used by the server to perform real-time anomaly detection using the trained stacking classifier as discussed before.

Both blacklisting and whitelisting are implemented by the CoAP server. Currently, devices can be added to the IoT network based on their type and OS. Only high complexity devices such as PCs, phones, laptops, and tablets are whitelisted in the simulation performed. However, more specific whitelist policies can easily be included for different scenarios, if needed. The CoAP server declines connection requests from devices which do not meet the whitelist policy described above after receiving information about the type and operating system of each device from I_C in message 7 of the connection phase of the handshake protocol (see Fig. 3). Consequently, the devices which satisfy the above conditions can generate a secure session with the CoAP server and are registered to the IoT network.

The network traffic generated by each individual IoT device follows a Poisson probability distribution, which is widely used for network and communication traffic simulations [56]. Factors such as packet collision, loss, and faulty packets are ignored during the network simulation for convenience. In a real network, collisions naturally occur when network traffic is high. This leads to longer delays and may prevent devices from entering the network. Then it may be difficult to measure the effectiveness of the protocol.

The Poisson distribution expresses the probability of a given number n of independently occurring events in a fixed duration of time τ , based on the average number of events λ observed per unit time:

$$f_n(\lambda) = \frac{(\lambda\tau)^n}{n!} \cdot e^{-\lambda\tau} \quad (4)$$

Table 7

Mean performance of the CoAP server using the Stacking classifier on the IoTID20 and XGBoost classifier on CICIDS-2018 datasets in a simulated network consisting of 7 benign devices and 5 malicious devices.

Dataset	Classification type	Role	Total packets	Misclassified	Accuracy
IoTID20	Binary	Victim	3871	66	98.29%
	Binary	Malicious	2720	1	99.96%
	Multi-class	Victim	3871	66	98.29%
	Multi-class	Malicious	2720	254	90.06%
CICIDS-2018	Binary	Victim	3892	46	98.82%
	Binary	Malicious	2730	194	92.89%
	Multi-class	Victim	3892	46	98.82%
	Multi-class	Malicious	2730	210	92.31%

Table 8

Latency of the phases of the security protocol with respect to TLS.

Protocol	Latency
Registration phase	280 ms
Connection phase	230 ms
TLS [55]	241 ms

Table 9

Number of high-low complexity devices with their roles in the network simulation.

Role	Device complexity	Number of devices
Victim	High	7
Victim	Low	18
Malicious	High	5

In this paper, the network simulation is based on the Poisson distribution with $\lambda = 3$ packets sent per second by each IoT device. This value of λ implies that each of the 30 IoT devices transmit 3 packets per second. This value is chosen so as to observe the network performance in more challenging traffic conditions. Note that λ can have any value as long as the CoAP server can handle the generated network traffic.

Due to the whitelisting policy, devices with lower complexity are denied connection by the server. Thus, only 12 devices can join the IoT network and begin generating network traffic. Assuming 12 devices join the IoT network, and each sends $\lambda = 3$ packets per second, there will be 36 packets per second on average. The average packet duration is 0.023 s as shown in Fig. 4. The collision probability between packets is given by $f_2(3) = 0.0011$ since collisions occur when we observe two or more packets during a packet duration, and the arrival probability of more than two packets is negligible. Thus, collisions between packets can be neglected in the simulation.

The CoAP server continuously monitors the network by feeding packet information to the classifier for real-time anomaly detection. The performance of the selected classifiers, Stacking for IoTID20 and XGBoost for CICIDS-2018, is presented in Table 7, evaluated in both binary and multi-class classification settings over both malicious and benign packets. In the binary setting, the classifiers effectively distinguish between benign and malicious traffic, achieving high accuracy with minimal misclassification. For example, the XGBoost classifier achieves strong binary classification performance on CICIDS-2018, while Stacking performs similarly on IoTID20.

In the multi-class setting, which involves identifying the specific type of attack, the classification task becomes more complex. Still, both classifiers maintain high accuracy. Although identifying the exact attack category is not essential for blacklisting purposes, the high multi-class accuracy further demonstrates the effectiveness of the anomaly detection framework. These results demonstrate that the proposed system can effectively identify and classify network threats, enabling the CoAP server to reliably blacklist malicious devices and maintain the security of the network.

We evaluate the communication cost for the anomaly detection model over a 180 second period, following the completion of the

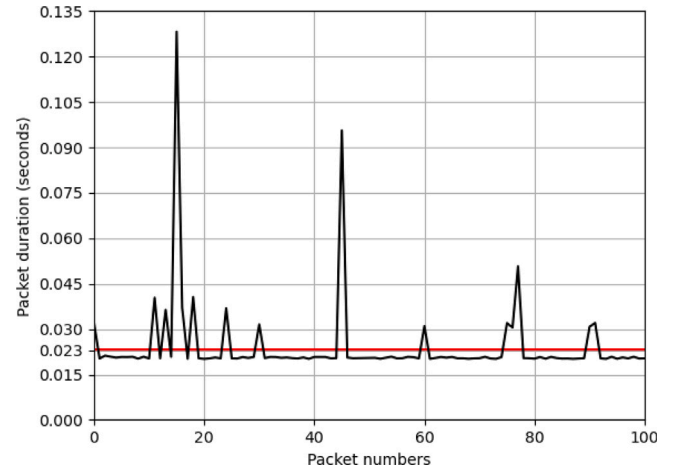


Fig. 4. Packet duration of the first 100 messages observed during the network simulation performed with 12 devices generating a network traffic of $\lambda = 3$ packets per second.

Table 10

Average communication cost analysis for the anomaly detection model over a 180-second period, following the completion of the registration phase. The analysis assumes only benign network traffic to prevent device blacklisting. The communication costs are broken down into device-side and server-side costs for the IoTID20 and the CICIDS-2018 datasets with varying numbers of devices.

Dataset used	# Devices	Device side	Server side
IoTID20	30	0.44 MB	13.53 MB
	60	0.45 MB	26.89 MB
	120	0.44 MB	53.56 MB
CICIDS-2018	30	0.16 MB	4.96 MB
	60	0.15 MB	10.13 MB
	120	0.15 MB	20.62 MB

registration phase. To prevent device blacklisting from affecting the analysis, we assumed only benign network traffic. For both datasets, Table 10 shows that both the server-side communication cost as well as the device side cost remains low, showing that the communication overhead for individual devices is minimal, thus supporting the scalability of the system.

Current limitations of the proposed system model include the assumption of secure verification between the IoT devices and the CA for server authentication. The model assumes secure communications between the two, which may be achieved through other handshake protocols like TLS or a long term pre-shared secret which is inserted into the IoT devices and the CA. Both assumptions have their downsides in the proposed model. TLS uses traditional session key generation and public key encryption, which requires high computational power and hence not suitable for all IoT devices. Similarly, using a long term pre-shared secret is not suitable for the proposed use case and

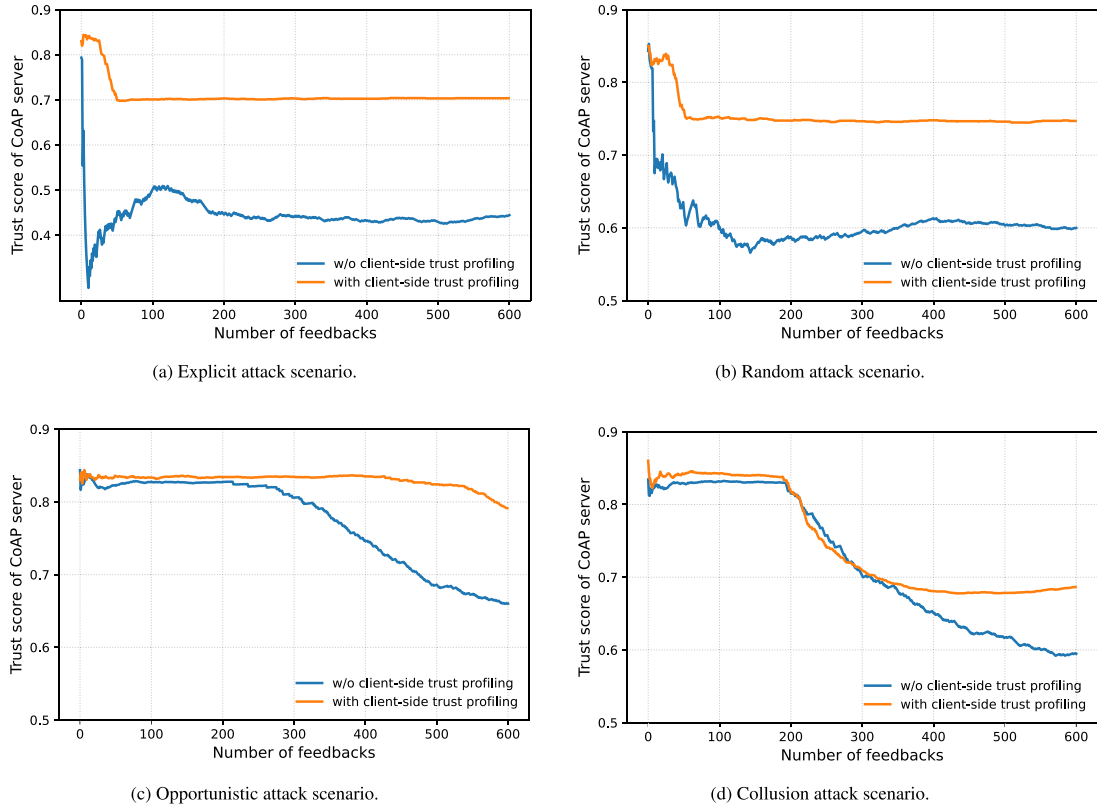


Fig. 5. Performance of client-side profiling performed by the smart contract on the trust score of the CoAP server.

system model. Since any device capable of completing the proposed security protocol may enter the IoT network, distributing the pre-shared secret to the new devices poses a challenge. The deficiencies of the proposed network simulation setup include the behavior of the malicious IoT nodes. The IoT nodes attacking the network generate high amounts of malicious network traffic, which makes it relatively easy for the server to detect them since the Stacking classifier used for autonomous anomaly detection has a high performance as discussed before. Nevertheless, this may not be the case in a real-life scenario where malicious devices actively try to impersonate benign devices by generating small amounts of malicious network traffic to avoid detection.

5.5. Network simulation of the trust model

A setup similar to that used to simulate the proposed security protocol is used to simulate the trust model. The setup includes a single CoAP server, the CA, the Ethereum blockchain network with a smart contract, and 30 IoT devices. It is to be noted that although a single CoAP server is used here for ease of simulation, the presented system is also valid for decentralized IoT networks with multiple CoAP servers. All IoT devices are assumed to be of high complexity to join the network due to the whitelisting policy discussed before. As in the security protocol, the simulation of the trust model does not assume previously known devices, so all the devices perform both phases of the security protocol to establish a secure session. Malicious devices in the network only perform behavior-based attacks so that the effectiveness of the trust model can be measured as optimally as possible. In this paper, a variety of malicious devices with different behaviors are considered and multiple simulations are performed to assess the performance of the trust provider and the resilience of the proposed system against a variety of trust-based attacks. Since we assume a benign server in the network simulation, all trust-based attacks attempt to badmouth the server, though the reverse can also be simulated successfully.

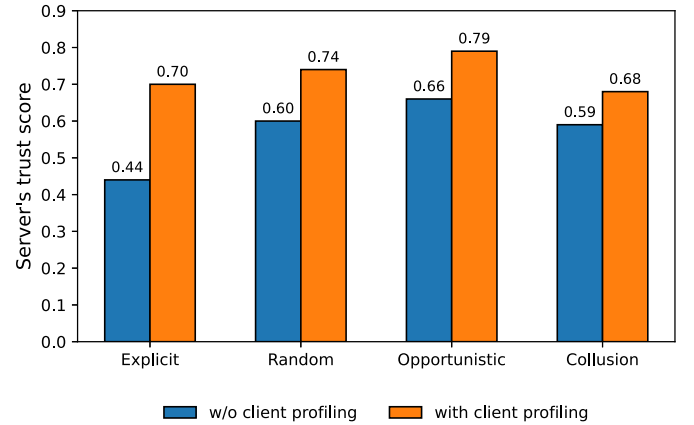


Fig. 6. Effect of client-side trust profiling on the trust of the CoAP server.

Table 11

Number of high-low complexity devices with their roles in the network simulation of the trust model.

Role	Device complexity	Number of devices
Victim	High	15
Malicious	High	15

The smart contract updates the trust of the CoAP server and the client nodes each time a new block is generated on the blockchain. In all network simulations, the ratio of benign and malicious clients is kept at 1:1 (half of the nodes in the network are benign and the other half are malicious) as shown in Table 11. Each client makes a total of 20 interactions with the CoAP server and writes a feedback on the blockchain after each interaction. We assume that the network traffic generated by all IoT devices is modeled by a Poisson distribution with

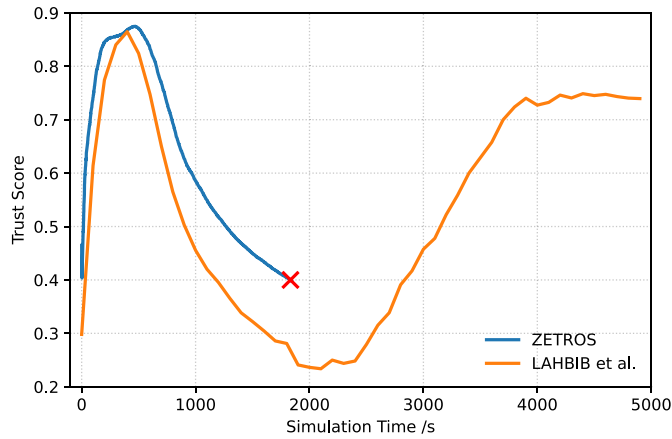


Fig. 7. Effect of blacklisting against a single malicious device performing a collusion attack.

the same parameters as the anomaly detection model, which eliminates packet collisions, packet losses, and faulty packets.

The effect of client-side trust profiling on the trust score of CoAP server may be observed in Fig. 5 for various trust-based attack scenarios. The trust score of the CoAP server without the client-side profiling is calculated by averaging the feedback data available on the blockchain. Whereas the trust score of the CoAP server with the client-side profiling is calculated by the smart contract using the evidence data stored on the Ethereum blockchain. The smart contract evaluates the honesty of each participating client inside the blockchain network autonomously after each feedback according to the consistency of the feedback and its evidence. The smart contract uses weighted averaging to calculate the trust score of the CoAP server.

Performance against explicit attacks is shown in Fig. 5(a), where malicious nodes bad-mouth on every interaction and do not care about their trust scores. Performance against random attacks is shown in Fig. 5(b), where malicious nodes randomly give inconsistent feedback, without considering their own trust score. Performance against opportunistic attacks is shown in Fig. 5(c), where malicious nodes try to attain a high trust score by behaving like benign nodes and giving consistent feedback. After obtaining a high trust score, they start acting maliciously. For the opportunistic attack scenario, attacker nodes score the first half of their interactions like benign nodes, and then perform bad-mouthing on the other half of their interactions. Performance against collusion attacks is shown in Fig. 5(d), where malicious nodes act together. They start out as benign nodes, but start acting maliciously in a coordinated way at the same time.

Our simulation results show that client-side trust profiling shows increased resilience against all trust-based attacks discussed in our adversary model as shown in Figs. 5 and 6. In all trust model simulations, the ratio of benign to malicious clients is maintained at 1:1, which is quite difficult for an attacker to achieve in a real-life network. Due to the addition of a client-side trust profile demonstrating increased resilience to trust system attacks, this approach can be used in both centralized and decentralized networks to increase the blacklisting accuracy of the server in systems similar to the one described in this paper.

We compare, in Fig. 7, the results of our trust scores with those found by the method presented in [36] for the case of a single malicious device performing a collusion attack. The collusion attack described in Section 4.2.3 is adopted here, where an initially benign device starts and stops acting maliciously throughout the simulation. According to [36], trust value is determined by Eq. (3) with $\Delta t = 1800$, which is the same value used in their simulations. Since [36] does not specify how w_1 and w_2 should be chosen, $w_1 = 0.1$ and $w_2 = 0.9$ are used, which further increases the significance of past trust values. In spite of this, Fig. 7 shows that if we allow the simulation to run for some

time, the malicious device obtains a high trust value based on the trust evaluation of [36]. When we employ blacklisting, our trust model, with a blacklisting threshold of 0.4, blacklists the malicious device when its trust value falls below the threshold for blacklisting.

6. Conclusions

This paper presents an IoT network security framework to provide security against protocol, network, and behavior-based attacks in a zero trust environment. A security protocol is proposed to provide communication privacy and mutual authentication between IoT clients and servers. An autonomous, distributed anomaly detection using ML algorithms is proposed to provide security against network-based attacks from inside the network. And finally, a trust management model utilizing smart contracts is proposed to increase resilience against behavioral attacks.

The protocol makes use of a PUF for session key generation, as well as whitelisting and blacklisting methods for ensuring the network security. To the best of our knowledge, the presented PUF-based handshake protocol is the first to perform blacklisting and whitelisting. Anomaly detection using ML techniques is utilized for blacklisting. An in-depth security analysis of the proposed handshake protocol is provided, where the resilience of the protocol against forgery, replay and MITM attacks is discussed. The paper also provides and discusses network traffic simulation carried out using the proposed handshake protocol. Through the collection of IoT node specifications, the CoAP server allows only those IoT nodes that meet certain specifications to join and remain in the IoT network. Additionally, a comparison of various ML algorithms performances and their fitness for real-time anomaly detection is provided over two real-life datasets. Amongst the six ML algorithms considered, the stacking and the XGBoost algorithm is shown to achieve the highest level of performance in detecting anomalous data in the network.

The proposed trust management model, based on Ethereum blockchain's and smart contracts, enables consistent autonomous trust scoring in distributed networks in a zero-trust environment. Using client-side trust profiling and average weighted trust scoring algorithm, the network simulations confirmed the resilience of the proposed trust system against various forms of trust attacks. The network simulation results show that our trust management model provides greater resilience to trust-based attacks using blacklisting compared to similar blockchain-based trust models in the literature.

Despite the promising results obtained in this study, several limitations of the proposed framework suggest opportunities for future research. Firstly, the current system assumes a secure communication channel between the IoT devices and the CA during the authentication phase. In environments where such secure verification cannot be guaranteed, future work could focus on removing this dependency by leveraging decentralized identity management mechanisms. Secondly, while the proposed anomaly detection and blacklisting model achieved high performance in simulated settings, it currently relies on pre-trained classifiers. Future research could explore adaptive learning or federated learning approaches to improve detection of emerging or evolving attack types in dynamic IoT environments. Furthermore, while the framework ensures verifiable and immutable feedback for trust scoring by using Ethereum-based DIDs, future work will explore stronger privacy-preserving mechanisms such as zero-knowledge proofs, differential privacy, or cryptographic anonymization schemes to further enhance trust computation without compromising user confidentiality.

CRedit authorship contribution statement

Cem Ata Baykara: Writing – review & editing, Writing – original draft, Visualization, Software, Methodology, Investigation, Conceptualization. **İlgin Şafak:** Writing – review & editing, Validation, Supervision, Resources, Project administration, Methodology, Funding acquisition, Conceptualization. **Kubra Kalkan:** Writing – review & editing, Validation, Supervision, Methodology, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by Business Finland (BF) within the EUREKACELTIC-NEXT project CISSAN.

Data availability

The authors do not have permission to share data.

References

- [1] A. Rayes, S. Salam, *Internet of Things from Hype to Reality*, Springer, 2019, pp. 1–35.
- [2] G. Santucci, The internet of things: Between the revolution of the internet and the metamorphosis of objects, *Vis. Challenges Realis. Internet Things* 1 (2010) 11–24.
- [3] T.D. Nguyen, S. Marchal, M. Miettinen, H. Fereidooni, N. Asokan, A.-R. Sadeghi, DIoT: A federated self-learning anomaly detection system for IoT, in: *IEEE 39th International Conference on Distributed Computing Systems, ICDCS, IEEE*, 2019, pp. 756–767.
- [4] G. Fortino, F. Messina, D. Rosaci, G.M. Sarnè, ResIoT: An IoT social framework resilient to malicious activities, *IEEE/CAA J. Autom. Sin.* 7 (5) (2020) 1263–1278.
- [5] S. Rose, O. Borchert, S. Mitchell, S. Connelly, *Zero Trust Architecture*, Tech. Rep., National Institute of Standards and Technology, 2020.
- [6] T. Frantti, I. Şafak, An architecture for enabling collective intelligence in IoT networks, in: *15th International Conference on Computational Collective Intelligence, ICCI*, 2023.
- [7] C.A. Baykara, I. Şafak, K. Kalkan, Shapeiot: secure handshake protocol for autonomous iot device discovery and blacklisting using physical unclonable functions and machine learning, *3rd Int. Conf. Mach. Learn. Appl.* 11 (15) (2021) 119–137.
- [8] D.F. Santos, H.O. Almeida, A. Perkusich, A personal connected health system for the internet of things based on the constrained application protocol, *Comput. Electr. Eng.* 44 (2015) 122–136.
- [9] T.A. Alghamdi, A. Lasebae, M. Aiash, Security analysis of the constrained application protocol in the internet of things, in: *Second International Conference on Future Generation Communication Technologies, FGCT, IEEE*, 2013, pp. 163–168.
- [10] D. Rathod, S. Patil, Security analysis of constrained application protocol (CoAP): IoT protocol, *Int. J. Adv. Stud. Comput. Sci. Eng.* 6 (8) (2017) 37.
- [11] Z. Shelby, K. Hartke, C. Bormann, The constrained application protocol (CoAP), 2022, https://iottestware.readthedocs.io/en/master/coap_rfc.html. (Accessed May 2022).
- [12] A. Shamsoshoara, A. Korenda, F. Afghah, S. Zeadally, A survey on physical unclonable function (PUF)-based security solutions for internet of things, *Comput. Netw.* 183 (2020) 107593.
- [13] G. Chen, F. Zeng, J. Zhang, T. Lu, J. Shen, W. Shu, An adaptive trust model based on recommendation filtering algorithm for the internet of things systems, *Comput. Netw.* 190 (2021) 107952.
- [14] S. Pal, M. Hitchens, V. Varadarajan, Towards the design of a trust management framework for the internet of things, in: *13th International Conference on Sensing Technology, ICST, IEEE*, 2019, pp. 1–7.
- [15] S. Pal, A. Hill, T. Rabehaja, M. Hitchens, A blockchain-based trust management framework with verifiable interactions, *Comput. Netw.* 200 (2021) 108506.
- [16] Y.L. Sun, Z. Han, W. Yu, K.R. Liu, Attacks on trust evaluation in distributed networks, in: *40th Annual Conference on Information Sciences and Systems, IEEE*, 2006, pp. 1461–1466.
- [17] M.U. Hassan, M.H. Rehmani, J. Chen, Privacy preservation in blockchain based IoT systems: Integration issues, prospects, challenges, and future research directions, *Future Gener. Comput. Syst.* 97 (2019) 512–529.
- [18] S. Nakamoto, *Bitcoin: A Peer-To-Peer Electronic Cash System*, Association of American Publishers (AAP), 2008, p. 21260.
- [19] W. Metcalfe, et al., *Ethereum, smart contracts, DApps*, *Blockchain Crypt Curr.* (2020) 77.
- [20] C. Núñez Gómez, C. Carrión, B. Caminero, F.M. Delicado, S-HIDRA: A blockchain and SDN domain-based architecture to orchestrate fog computing environments, *Comput. Netw.* (2022) 109512.
- [21] Y. Yilmaz, S.R. Gunn, B. Halak, Lightweight PUF-based authentication protocol for IoT devices, in: *IEEE 3rd International Verification and Security Workshop, IVSW, IEEE*, 2018, pp. 38–43.
- [22] A. Bhattacharyya, T. Bose, S. Bandyopadhyay, A. Ukil, A. Pal, LESS: Lightweight establishment of secure session: A cross-layer approach using CoAP and DTLS-PSK channel encryption, in: *IEEE 29th International Conference on Advanced Information Networking and Applications Workshops, IEEE*, 2015, pp. 682–687.
- [23] A.B. Rabbiah, K. Ramakrishnan, E. Liri, K. Kar, A lightweight authentication and key exchange protocol for IoT, in: *Workshop on Decentralized IoT Security and Standards*, vol. 2018, 2018, pp. 1–6.
- [24] J.W. Byun, End-to-end authenticated key exchange based on different physical unclonable functions, *IEEE Access* 7 (2019) 102951–102965.
- [25] J.W. Byun, A generic multifactor authenticated key exchange with physical unclonable function, *Secur. Commun. Netw.* 2019 (2019).
- [26] M.N. Aman, K.C. Chua, B. Sikdar, Mutual authentication in IoT systems using physical unclonable functions, *IEEE Internet Things J.* 4 (5) (2017) 1327–1340.
- [27] A. Braeken, PUF based authentication protocol for IoT, *Symmetry* 10 (8) (2018) 352.
- [28] U. Chatterjee, et al., PUF+ IBE: blending physically unclonable functions with identity based encryption for authentication and key exchange in IoTs, *Cryptol. EPrint Arch.* (2017).
- [29] N. Li, D. Liu, S. Nepal, Lightweight mutual authentication for IoT and its applications, *IEEE Trans. Sustain. Comput.* 2 (4) (2017) 359–370.
- [30] Y. Ma, L. Yan, X. Huang, M. Ma, D. Li, DTLSShps: SDN-based DTLS handshake protocol simplification for IoT, *IEEE Internet Things J.* 7 (4) (2020) 3349–3362.
- [31] J. Park, N. Kang, Lightweight secure communication for coop-enabled internet of things using delegated DTLS handshake, in: *International Conference on Information and Communication Technology Convergence, ICTC, IEEE*, 2014, pp. 28–33.
- [32] B. Anderson, D. McGrew, Accurate TLS fingerprinting using destination context and knowledge bases, 2020, arXiv preprint arXiv:2009.01939.
- [33] S.A. Ghasempouri, B.T. Ladani, Modeling trust and reputation systems in hostile environments, *Future Gener. Comput. Syst.* 99 (2019) 571–592.
- [34] S.K. Prajapati, S. Changder, A. Sarkar, Trust management model for cloud computing environment, 2013, arXiv preprint arXiv:1304.5313.
- [35] S.E.A. Rafeey, A. Abdel-Hamid, M. Abou El-Nasr, CBSTm-IoT: Context-based social trust model for the internet of things, in: *International Conference on Selected Topics in Mobile & Wireless Networking, MoWNeT, IEEE*, 2016, pp. 1–8.
- [36] A. Lahbib, K. Toumi, A. Laouiti, A. Laube, S. Martin, Blockchain based trust management mechanism for IoT, in: *2019 IEEE Wireless Communications and Networking Conference, WCNC, IEEE*, 2019, pp. 1–8.
- [37] D. Dolev, A. Yao, On the security of public key protocols, *IEEE Trans. Inform. Theory* 29 (2) (1983) 198–208.
- [38] G. Kambourakis, C. Kolias, A. Stavrou, The mirai botnet and the iot zombie armies, in: *MILCOM IEEE Military Communications Conference, MILCOM, IEEE*, 2017, pp. 267–272.
- [39] S. Byers, A.D. Rubin, D. Kormann, Defending against an internet-based attack on the physical world, *ACM Trans. Internet Technol. (TOIT)* 4 (3) (2004) 239–254.
- [40] J.M. Estevez-Tapiador, P. García-Teodoro, J.E. Díaz-Verdejo, Detection of web-based attacks through Markovian protocol parsing, in: *10th IEEE Symposium on Computers and Communications, ISCC'05, IEEE*, 2005, pp. 457–462.
- [41] R. Xu, W.-l. Ma, W.-l. Zheng, Defending against UDP flooding by negative selection algorithm based on eigenvalue sets, in: *Fifth International Conference on Information Assurance and Security*, vol. 2, IEEE, 2009, pp. 342–345.
- [42] A. Sridharan, T. Ye, S. Bhattacharyya, Connectionless port scan detection on the backbone, in: *IEEE International Performance Computing and Communications Conference, IEEE*, 2006, pp. 10–pp.
- [43] P.F. Tehrani, E. Osterweil, T.C. Schmidt, M. Wählisch, A security model for web-based communication, *Commun. ACM* 67 (10) (2024) 83–90.
- [44] T. Jager, F. Kohlar, S. Schäge, J. Schwenk, Authenticated confidential channel establishment and the security of TLS-DHE, *J. Cryptology* 30 (2017) 1276–1324.
- [45] C. Krähenbühl, A. Perrig, Ubiquitous secure communication in a future internet architecture, *SN Comput. Sci.* 3 (5) (2022) 350.
- [46] T. Mehmood, H.B.M. Rais, Machine learning algorithms in context of intrusion detection, in: *3rd International Conference on Computer and Information Sciences, ICCOINS, IEEE*, 2016, pp. 369–373.
- [47] P. Illavarasan, B.K. Sundaram, A study of intrusion detection system using machine learning classification algorithm based on different feature selection approach, in: *Third International Conference on I-SMAC, IoT in Social, Mobile, Analytics and Cloud, I-SMAC, IEEE*, 2019, pp. 295–299.
- [48] A. Srivastava, A. Agarwal, G. Kaur, Novel machine learning technique for intrusion detection in recent network-based attacks, in: *4th International Conference on Information Systems and Computer Networks, ISCON, IEEE*, 2019, pp. 524–528.
- [49] I. Ullah, Q.H. Mahmoud, A scheme for generating a dataset for anomalous activity detection in iot networks, in: *Canadian Conference on Artificial Intelligence, Springer*, 2020, pp. 508–520.
- [50] I. Sharafaldin, A.H. Lashkari, A.A. Ghorbani, et al., Toward generating a new intrusion detection dataset and intrusion traffic characterization, *ICISSp* 1 (2018) 108–116.

- [51] uPort Project, Ethr-DID library, 2025, <https://github.com/uport-project/ethr-did.git>. (Accessed 28 June 2025).
- [52] E. Anacleto, Machine learning for intrusion detection on CICIDS2017, 2019, <https://github.com/anacleto/ml-intrusion-detection-cicids2017>. (Accessed 08 April 2025).
- [53] A. Starzacher, B. Rinner, Evaluating KNN, LDA and QDA classification for embedded online feature fusion, in: International Conference on Intelligent Sensors, Sensor Networks and Information Processing, IEEE, 2008, pp. 85–90.
- [54] T.G. Tanganelli, CoAPthon v4.0, 2017, <https://github.com/Tanganelli/CoAPthon>.
- [55] L. Zhu, J. Amann, J. Heidemann, Measuring the latency and pervasiveness of TLS certificate revocation, in: International Conference on Passive and Active Network Measurement, Springer, 2016, pp. 16–29.
- [56] B. Chandrasekaran, Survey of network traffic models, Washington Univ. St. Louis CSE 567 (2009).